ToddySoft Connect S7 Web API Driver - Ignition 8.3

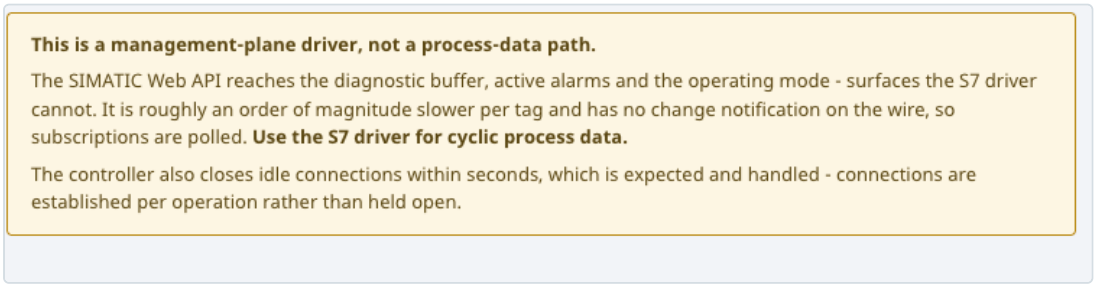
English version: README.md

Ignition-8.3-Modul zum Zugriff auf Siemens-Steuerungen S7-1200 und S7-1500 über die **SIMATIC Web API** — die HTTPS-/JSON-RPC-Schnittstelle der Steuerung selbst.

> **Hinweis zur Sprache:** Bedienoberfläche, Feld- und Abschnittsnamen sowie Menüpfade sind in Ignition durchgängig **englisch**. Diese Begriffe (z. B. *Host*, *Web API Session*, *Poll Interval*) bleiben daher auch in dieser deutschen Anleitung unverändert englisch, damit sie zur Oberfläche passen.

> **Dies ist ein Treiber für die Management-Ebene, kein Weg für Prozessdaten.**
>
> Die Web API erreicht Diagnosepuffer, aktive Alarme und die Betriebsart — Bereiche, die der S7-Treiber nicht erreicht. Im Gegenzug ist sie je Tag etwa **um den Faktor zehn langsamer** und kennt **keine Änderungsbenachrichtigung auf der Leitung**, jede Subscription wird also gepollt.
>
> **Verwenden Sie für zyklische Prozessdaten das S7-Modul.** Dieses Modul ist für Diagnose und Nicht-Tag-Bereiche gedacht oder für Umgebungen, in denen die Netzwerkrichtlinie ausschließlich HTTPS zulässt. Derselbe Hinweis steht oben im Konfigurationsformular des Geräts, denn dort ist die Entscheidung zum letzten Mal günstig zu ändern.

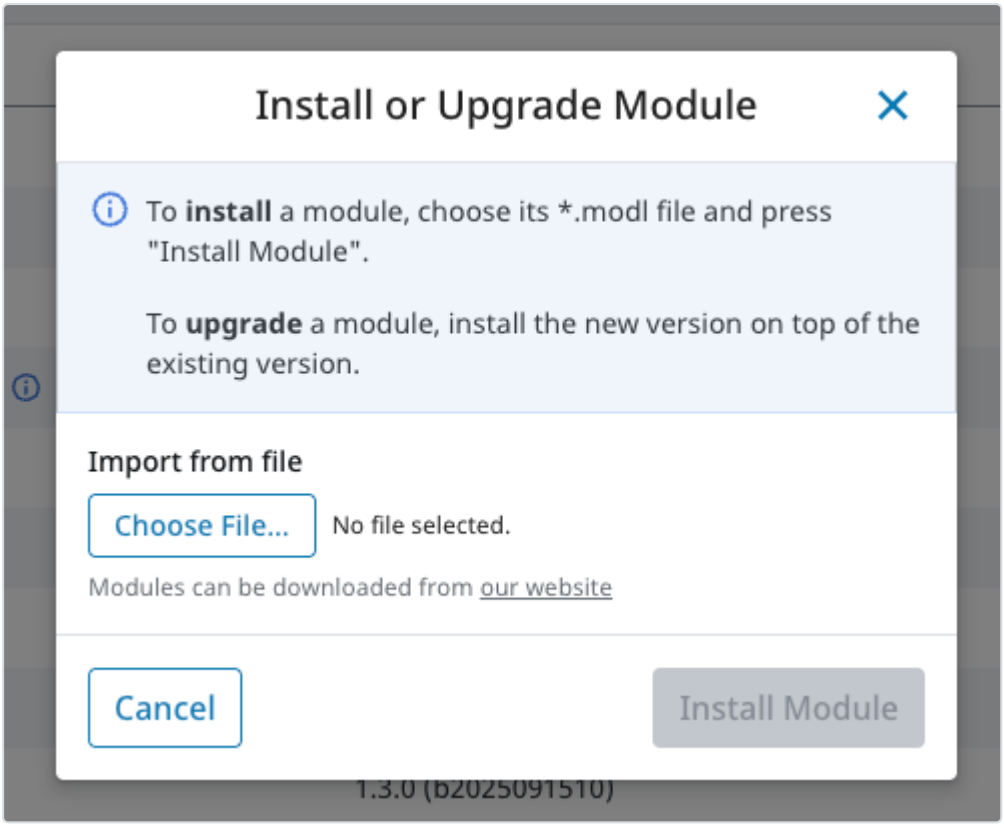
Die Steuerung schließt zudem untätige Verbindungen binnen Sekunden. Das ist erwartet und wird behandelt — der Treiber baut Verbindungen je Vorgang auf, statt eine offen zu halten. Ein Gerät, das in den Logs „ständig neu verbindet", arbeitet also wie vorgesehen.



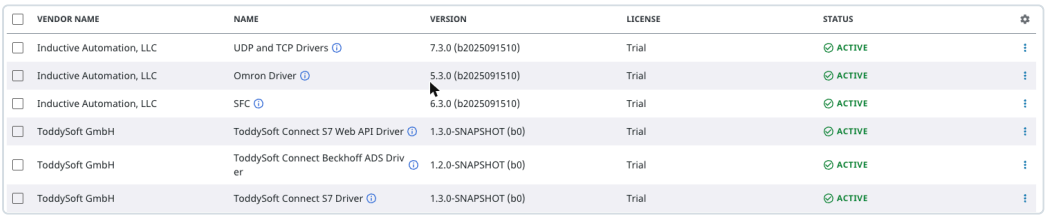
Installation

> **Dieses Modul ist ausschließlich für Ignition 8.3.** Es wird gegen das Ignition-8.3-SDK gebaut und läuft auf keiner anderen Hauptversion. Für Ignition 8.1 ist der separate Build `…-8.1.modl` zu installieren — die beiden sind nicht austauschbar, jedes Gateway braucht den passenden Build.
>
> **Ignition 8.0 und älter werden nicht unterstützt und können es auch nicht sein.** Das Modul baut auf der Gerätetreiber-API auf, die Ignition erst mit 8.1 eingeführt hat; auf 8.0 schlägt das Laden mit `NoClassDefFoundError: … opcua.server.api.AbstractDeviceModuleHook` fehl.
>
> Beide Builds tragen im Modul-Verzeichnis des Gateways denselben Namen; unterscheiden lassen sie sich an der installierten Datei oder an der Modulbeschreibung, die die Ignition-Version nennt.
>
> **Das OPC-UA-Modul von Ignition muss installiert sein und laufen.** Dies ist ein OPC-UA-Gerätetreiber: Er registriert sich am OPC-UA-Server von Ignition, und jeder Tag, den er bereitstellt, wird über diesen ausgeliefert. Fehlt das Modul, kann das Gateway das vorliegende nicht laden; im Log häufen sich dann `ClassNotFoundException` bzw. `NoClassDefFoundError` zu Klassen aus `com.inductiveautomation.opcua…`. Das OPC-UA-Modul gehört zum Lieferumfang von Ignition und ist standardmäßig installiert; praktisch relevant wird das daher nur auf Gateways, auf denen es entfernt oder deaktiviert wurde. Dieses Modul deklariert die Abhängigkeit als **required** — ein Mechanismus, den 8.3 besitzt und 8.1 nicht. Ignition versucht daher gar nicht erst, es zu laden, und nennt stattdessen den Grund, statt mit einem Stacktrace zu scheitern. Unter **Status > Diagnostics > Logs** erscheint:
>
> `Module "com.toddysoft.connect.…" depends on "com.inductiveautomation.opcua", which is either not installed or inactive on this gateway.`
>
> Die Treiber von Inductive Automation werden in derselben Situation genauso behandelt.

1. Öffnen Sie die Weboberfläche des Ignition Gateways und navigieren Sie zu **Config > System > Modules**.
2. Klicken Sie auf **Install or Upgrade a Module...**.
3. Wählen Sie die Datei `ToddySoft-Connect-S7-Web-API-Driver-8.3.modl` aus und laden Sie sie hoch.



4. Bestätigen Sie die Lizenz-/Zertifikatsabfrage, um die Installation fortzusetzen.
5. Warten Sie, bis das Gateway fertig ist, und prüfen Sie, ob das Modul in der Modulliste als **Running** angezeigt wird.



VENDOR NAME	NAME	VERSION	LICENSE	STATUS
Inductive Automation, LLC	UDP and TCP Drivers	7.3.0 (b2025091510)	Trial	ACTIVE
Inductive Automation, LLC	Omron Driver	5.3.0 (b2025091510)	Trial	ACTIVE
Inductive Automation, LLC	SFC	6.3.0 (b2025091510)	Trial	ACTIVE
ToddySoft GmbH	ToddySoft Connect S7 Web API Driver	1.3.0-SNAPSHOT (b0)	Trial	ACTIVE
ToddySoft GmbH	ToddySoft Connect Beckhoff ADS Driver	1.2.0-SNAPSHOT (b0)	Trial	ACTIVE
ToddySoft GmbH	ToddySoft Connect S7 Driver	1.3.0-SNAPSHOT (b0)	Trial	ACTIVE

Vorbereitung auf der Steuerung

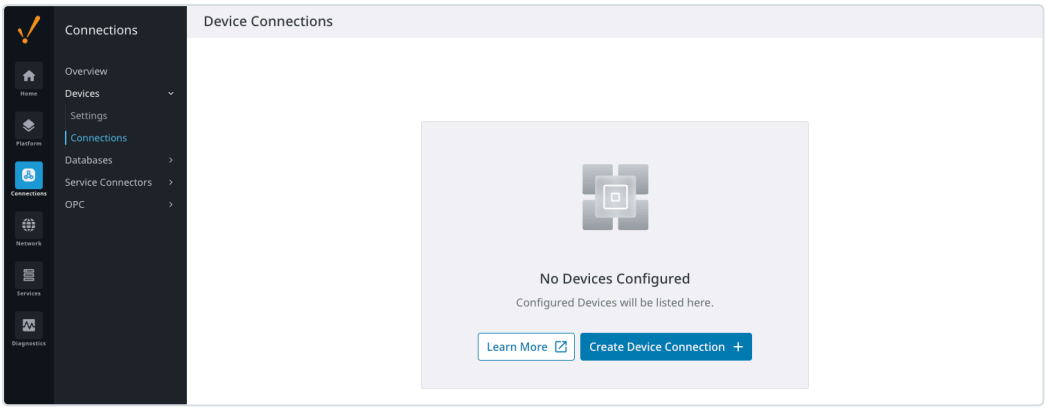
Zwei Voraussetzungen müssen auf der Steuerung erfüllt sein, bevor sich ein Gerät hier verbinden kann — und keine davon kann dieses Modul für Sie herstellen:

- **Die Web API muss für die CPU im TIA-Projekt aktiviert sein** (*Schutz & Security* > *Webserver*), und der Webserver selbst muss eingeschaltet sein.
- **Das Konto muss die Web-API-Berechtigung besitzen.** Ein Benutzer, der sich an den Webseiten der Steuerung anmelden kann, darf die API nicht automatisch nutzen; die Berechtigung wird im TIA-Projekt gesondert vergeben.

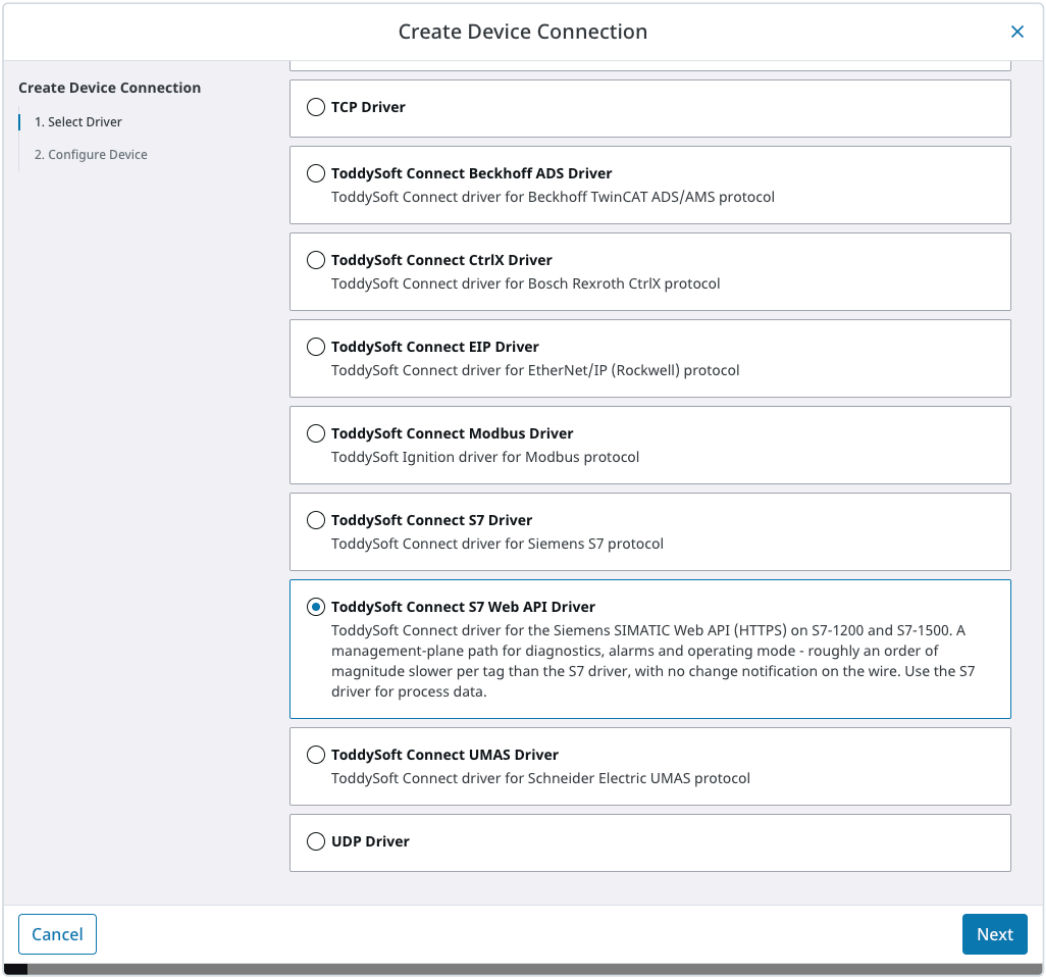
Fehlt Letztere, gelingt die Verbindung auf TLS-Ebene und die Anmeldung wird anschließend abgewiesen — was sich wie ein falsches Passwort liest.

Erstellen einer Geräteverbindung (Device Connection)

1. Gehen Sie zu **Config > OPC UA > Device Connections**.

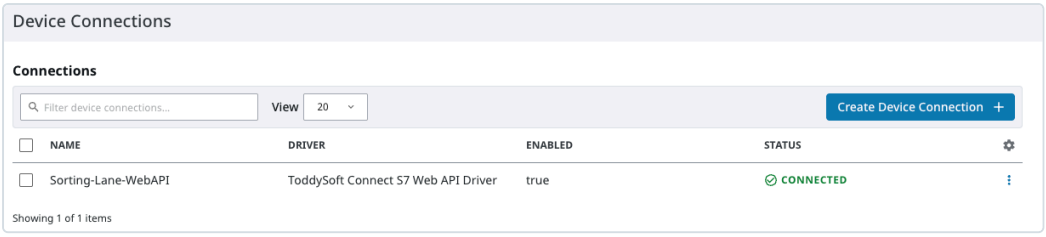


2. Klicken Sie auf **Create new Device...** und wählen Sie **ToddySoft Connect S7 Web API Driver** als Gerätetyp.



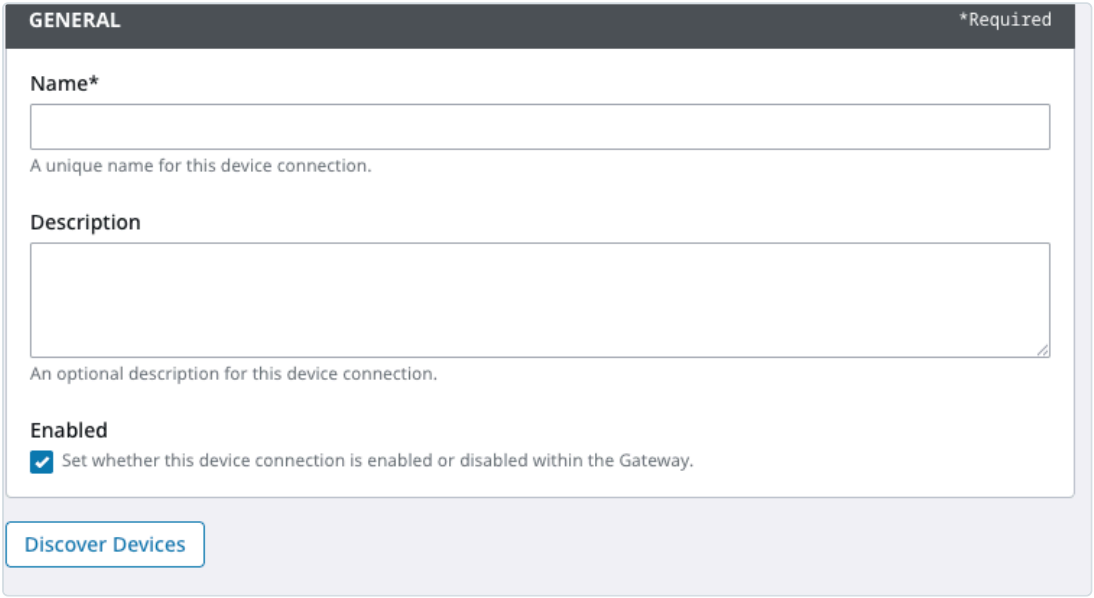
3. Konfigurieren Sie die Verbindungseinstellungen (siehe die Abschnitte unten) und klicken Sie auf **Create Device Connection**.

4. Die neue Verbindung erscheint anschließend in der Liste der Device Connections.



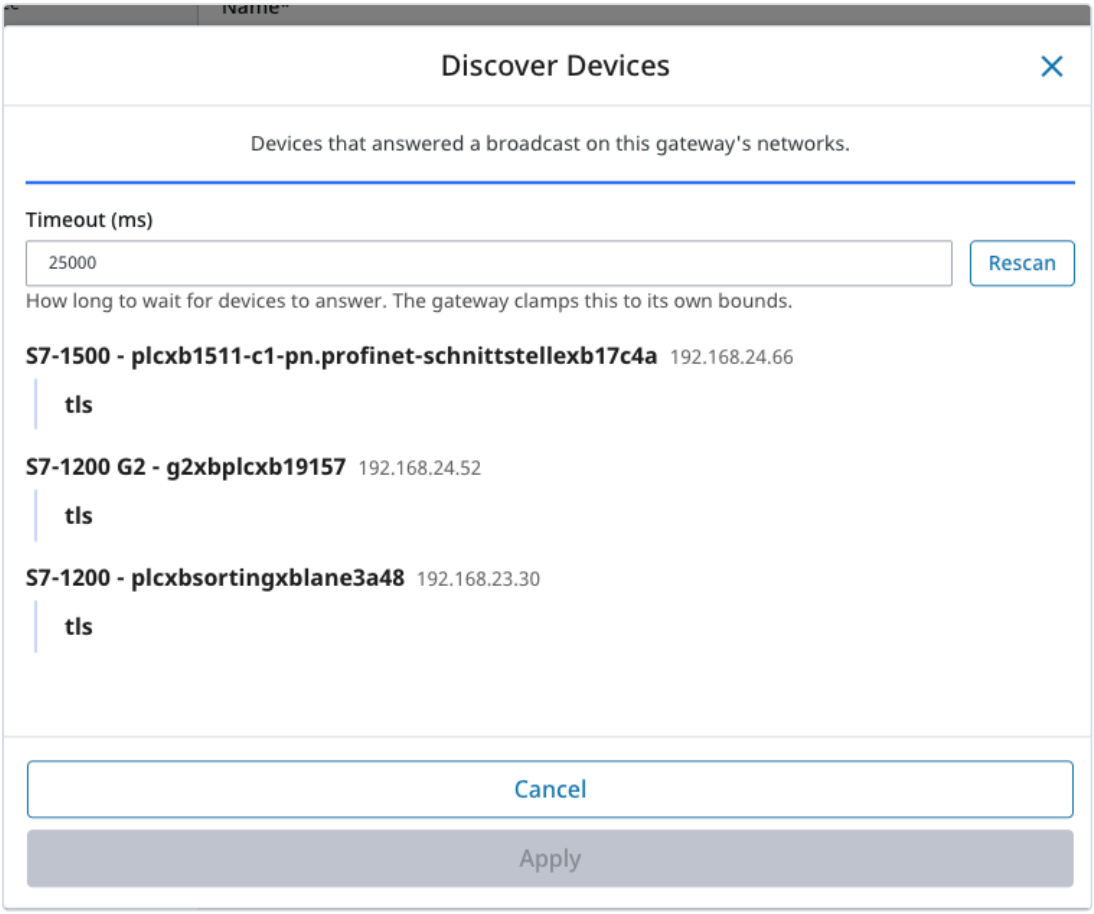
Geräte im Netzwerk finden

Beim Anlegen eines Geräts trägt das Konfigurationsformular direkt unter dem Abschnitt **General** — oberhalb der ersten Einstellungskarte — eine Schaltfläche **Discover Devices**.



Discover Devices

Durchsucht die Netzwerke des Gateways nach Steuerungen und fragt anschließend jede antwortende Steuerung, ob sie tatsächlich die Web API bedient, bevor sie aufgeführt wird. 443 mit etwas anderem antwortet, erscheint nicht, und eine, die den Test nicht rechtzeitig beantwortet, ebenfalls nicht — das Ergebnis ist also die Menge der Steuerungen, mit denen sich dieser Treiber wirklich verbinden kann, und nicht alles mit offenem HTTPS-Port.



Eine Steuerung, die auf

Die Auswahl eines Ergebnisses füllt **Host** und **Port**. Zwei Dinge tut sie bewusst nicht:

- **Sie trägt nur ein, was das Gerät tatsächlich gemeldet hat.** Ein Feld, zu dem der Suchlauf nichts gesagt hat, bleibt unangetastet statt geraten zu werden.
- **Sie trägt niemals Benutzernamen oder Passwort ein.** Ein Broadcast sagt, *wo* eine Steuerung ist, niemals welches Konto ihre API nutzen darf — die Web-API-Berechtigung wird je Konto im TIA-Projekt vergeben, es gibt also nichts sinnvoll zu erraten.

Timeout (ms) begrenzt, wie lange das Gateway zuhört; der Standardwert ist **25000**. Der Suchlauf selbst dauert rund fünfzehn Sekunden, und der Test je Steuerung wartet zehn Sekunden, bevor er einen langsamen TLS-Handshake aufgibt. Eine kürzere Grenze bricht den Suchlauf ab, bevor die Antworten eintreffen, und die Anzeige meldet dann *No devices answered* über einem Netz voller Steuerungen.

Würden durch das Übernehmen eines Ergebnisses bereits eingetragene Werte überschrieben, werden diese aufgelistet und es wird vorher nachgefragt. Ein Suchlauf erfordert die Berechtigung, Geräte verbindungen anzulegen; angemeldet zu sein genügt nicht.

> **In Docker braucht die Gerätesuche den Discovery-Agenten.** Ein Container liegt im Docker-Netz und nicht im Anlagennetz der Steuerungen, die Suche allein findet dort also nichts. Siehe Betrieb in Docker: der Discovery-Agent; ohne ihn tragen Sie den Host von Hand ein — alles andere auf dieser Seite funktioniert normal.

> **Dieser Treiber bietet keine Schaltfläche *Test Connection*.** Diese Aktion erscheint nur bei Treibern, die eine vom Gateway prüfbare Device-Setup-Fähigkeit veröffentlichen — dieser tut das nicht. Die Schaltfläche fehlt, statt wirkungslos vorhanden zu sein. Das Beckhoff-ADS-Modul bietet sie an.

Betrieb in Docker: der Discovery-Agent

Ein Gateway in einem Docker-Container sieht nur die Netze des Containers, nicht die der Anlage: der PROFINET-DCP-Broadcast hinter *Discover Devices* verlässt das Netz des Containers nie. Hinzu kommt, dass das offizielle Docker-Image von Ignition kein libpcap enthält, mit dem die Suche ihre Rahmen sendet und mitschneidet; im Container kann sie also gar nicht laufen und meldet, dass nichts geantwortet hat. Der **ToddySoft Discovery-Agent** schließt diese Lücke. Er ist ein kleiner, eigener Container, der auf derselben Maschine im Host-Netzwerk läuft und die Anlagennetze im Auftrag des Gateways durchsucht. Der Agent sucht nur: Verbinden, Durchsuchen und Lesen bleiben beim Gateway und laufen über das normale Routing der Maschine. Ein Gateway, das direkt auf der Anlagenmaschine installiert ist oder als Container im Host-Netzwerk läuft, braucht ihn nicht.

Installation. Der Agent wird als Image-Archiv zusammen mit diesem Modul geliefert, in derselben Freigabe, aus der Sie das Modul herunterladen: `discovery-agent-<Version>-<Architektur>.tar.gz`, für `amd64` oder `arm64`. Verwenden Sie den Agenten in der Version des Moduls. Auf der Maschine, auf der der Ignition-Container läuft (Linux):

```
docker load < discovery-agent-1.3.0-amd64.tar.gz
docker run -d --name toddysoft-discovery-agent \
  --restart unless-stopped --network host --cap-add NET_RAW \
  toddysoft/discovery-agent:1.3.0
```

`--network host` lässt ihn in den Anlagennetzen suchen, `NET_RAW` erlaubt ihm, die Suchrahmen zu senden und mitzuschneiden. Docker Desktop (macOS, Windows) betreibt Container in einer virtuellen Maschine; der Agent würde deren Netze durchsuchen statt die der Anlage.

In Ignition ist nichts einzustellen. Bei jedem **Discover Devices** sucht das Modul den Agenten am Default-Gateway seines Containers, unter Docker also auf der Maschine selbst. Ein später gestarteter Agent wird bei der nächsten Suche gefunden, ohne Neustart. Eine Zeile über den Ergebnissen sagt, was geschehen ist:

- *via discovery agent*: Der Agent hat gesucht; seine Geräte stehen zusammen mit denen, die das Gateway selbst gefunden hat, jedes einmal in der Liste.
- *no discovery agent found · container networks only*: Es läuft kein Agent; die Suche ist wie bisher die des Gateways.
- *discovery agent refused: …* oder *discovery agent not usable: …*: Der Grund folgt, etwa die Lizenz oder das gemeinsame Geheimnis (siehe unten).

Das Panel zeigt seine Ergebnisse nach etwa fünf Sekunden, sobald der Agent fertig ist. Die aufgeführten Geräte kommen vom Agenten: Die Suche des Gateways selbst kann im Container nicht laufen (siehe oben), was das Gateway-Log als *this gateway's own search could not run … the discovery agent answered* vermerkt. Antwortet kein Agent, nennt das Panel den Grund unter *Part of the search failed*.

Lizenz. Der Agent sucht nur für ein Gateway, das für diesen Treiber lizenziert ist. Mit jeder Suche sendet das Modul die Lizenz, mit der es läuft, die eingebaute Testlizenz eingeschlossen, über eine verschlüsselte Verbindung; der Agent prüft sie, verwendet sie für diese eine Suche und speichert nichts.

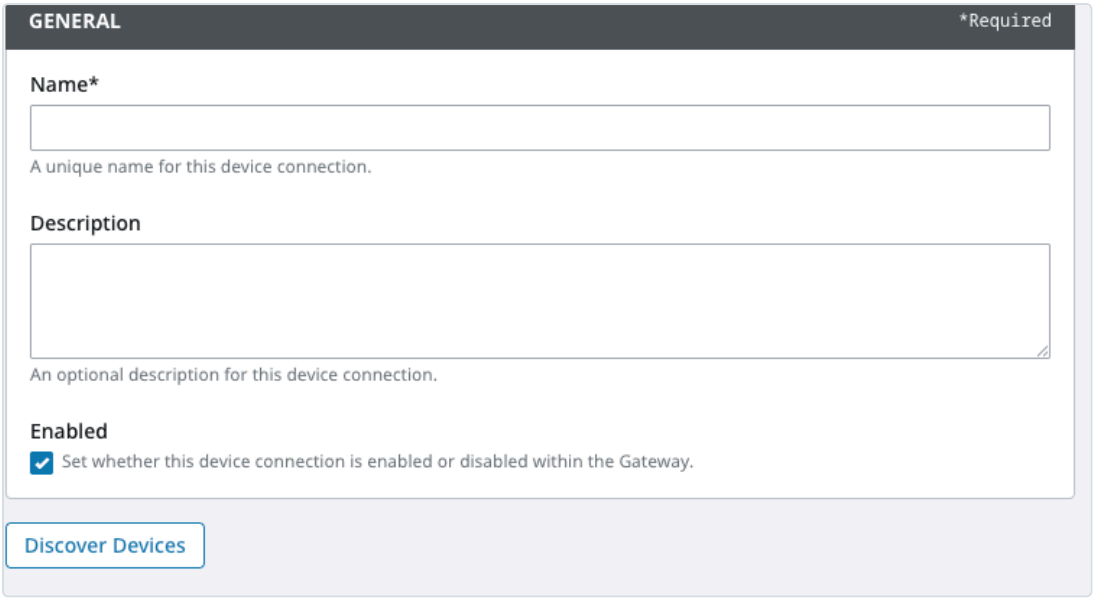
Sicherheit. Der Agent lauscht nur auf den eigenen Adressen der Docker-Netze auf dieser Maschine (Port 47900), nie in einem Anlagennetz, und nimmt Verbindungen nur aus den Container-Netzen an. Wer ihn weiter einschränken will, gibt ihm ein gemeinsames Geheimnis: `-e TODDYSOFT_AGENT_SECRET=<Geheimnis>` am Agenten und `-e TODDYSOFT_DISCOVERY_AGENT_SECRET=<Geheimnis>` am Ignition-Container. Ist ein Geheimnis gesetzt, wird ein Gateway ohne dieses oder mit einem anderen abgewiesen. Soll das Modul gar nicht nach einem Agenten suchen, setzen Sie `TODDYSOFT_DISCOVERY_AGENT=off` am Ignition-Container.

Konfiguration

Das Konfigurationsformular ist von oben nach unten in folgende Abschnitte gegliedert: **General**, **Connection**, **Web API Session**, **Certificate Trust**, **Timing**, **Limits** und **Audit Log**.

General

Vergibt Name und optionale Beschreibung der Verbindung und steuert, ob sie aktiv ist.



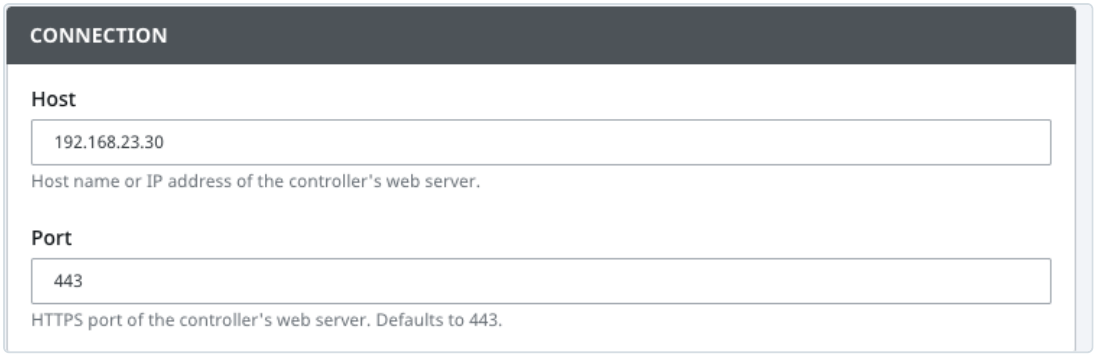
Einstellung	Beschreibung	Standard
Name	Eindeutiger Name dieser Geräteverbindung	
Description	Optionale Beschreibung dieser Verbindung	
Enabled	Ob die Verbindung aktiv ist	`true`

Ist `Enabled` nicht gesetzt, wird keine Verbindung aufgebaut, bis die Verbindung aktiviert wird.

Connection

Einstellung	Beschreibung	Standard
Host	Hostname oder IP-Adresse des Webservers der Steuerung	`localhost`
Port	HTTPS-Port des Webservers der Steuerung	`443`

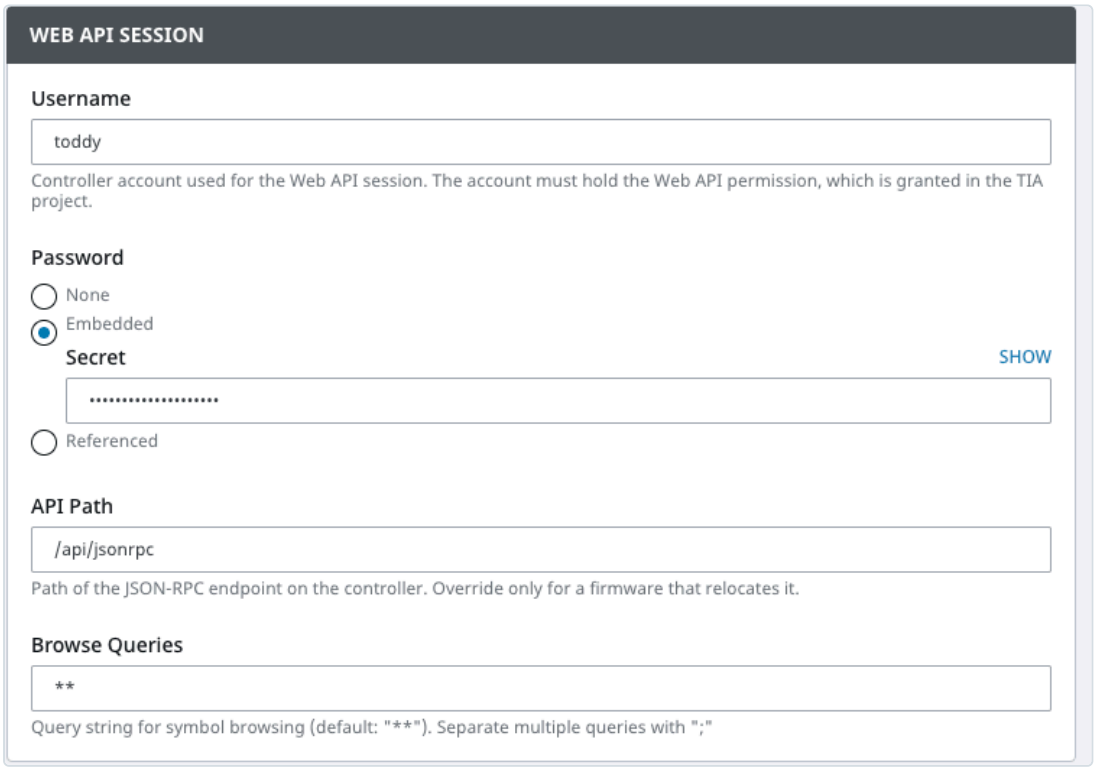
Die Web API ist ausschließlich HTTPS. Es gibt keinen Klartextmodus als Rückfallebene und keinen Transport zur Auswahl; eine reine HTTP-Verbindung wird abgewiesen. TLS ist der einzige Transport dieses Treibers — deshalb stehen die zugehörigen Einstellungen unter *Certificate Trust* und nicht hinter einer Transport-Auswahl.



Web API Session

Einstellung	Beschreibung	Standard
Username	Konto der Steuerung für die Web-API-Sitzung	
Password	Passwort des Web-API-Kontos. Wird in allen Logausgaben unkenntlich gemacht	*None*
API Path	Pfad des JSON-RPC-Endpunkts auf der Steuerung	`/api/jsonrpc`
Browse Queries	Filter für das Browsen von Symbolen. Mehrere Abfragen mit `;` trennen	`**`

Anonymen Zugriff gibt es nicht — die API authentifiziert jede Anfrage, ein Benutzername ist daher erforderlich. Das Konto muss die im TIA-Projekt vergebene **Web-API-Berechtigung** besitzen; Zugriff auf den Webserver allein genügt nicht.



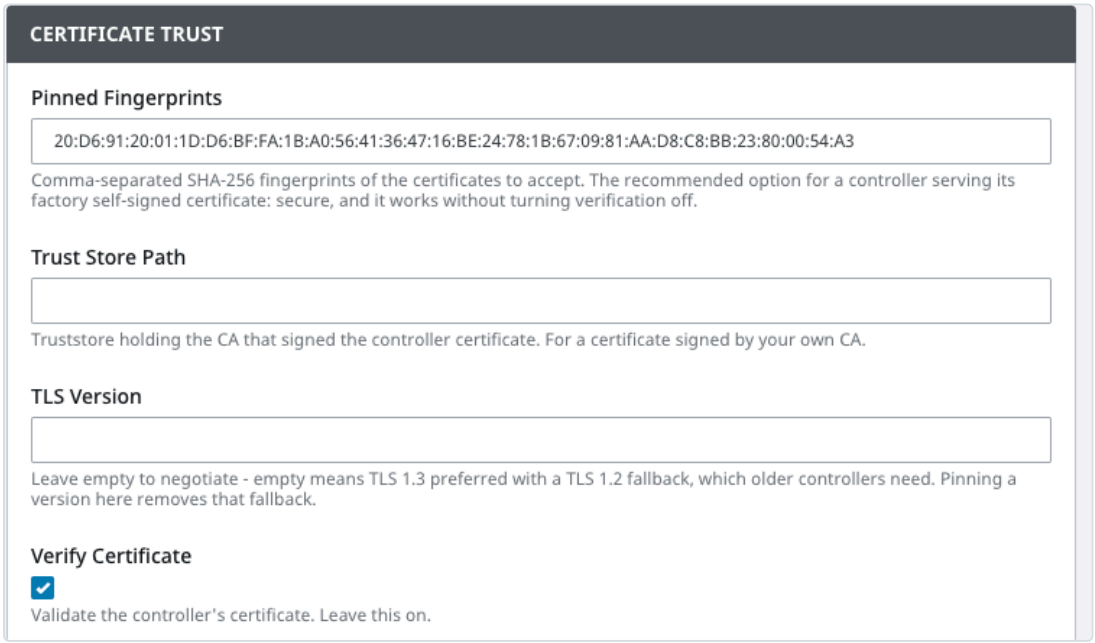
API Path ist nur für eine Firmware zu ändern, die den Endpunkt verlegt. Für jede aktuelle S7-1200-/S7-1500-Firmware ist der Standardwert richtig.

Seit ToddySoft Connect 1.2.0 werden die Zugangsdaten dem Treiber als Objekt übergeben und nicht mehr in die Verbindungszeichenfolge geschrieben. Sie erreichen damit weder das Gateway-Log noch ein Support-Bundle noch irgendeine Zeichenfolge, die die Verbindung zitiert.

Certificate Trust

Einstellung	Beschreibung	Standard
Verify Certificate	Das TLS-Zertifikat der Steuerung prüfen	`true`
Pinned Fingerprints	Kommagetrennte SHA-256-Fingerprints der zu akzeptierenden Zertifikate	*(leer)*
Trust Store Path	Trust Store mit der CA, die das Steuerungszertifikat signiert hat	*(leer)*
Trust Store Password	Passwort des Trust Stores. Für einen ungeschützten Store auf *None* lassen	*None*
Trust Store Type	Format des Trust Stores: `PKCS12` oder `JKS`	`PKCS12`
TLS Version	Protokollversion festlegen, z. B. `TLSv1.3`. Leer lassen für Aushandlung	*(leer)*

Eine Steuerung ab Werk liefert ein selbstsigniertes Zertifikat aus, dieser Entscheidung können Sie also nicht ausweichen. Die Reihenfolge, in der der Treiber vorgeht:



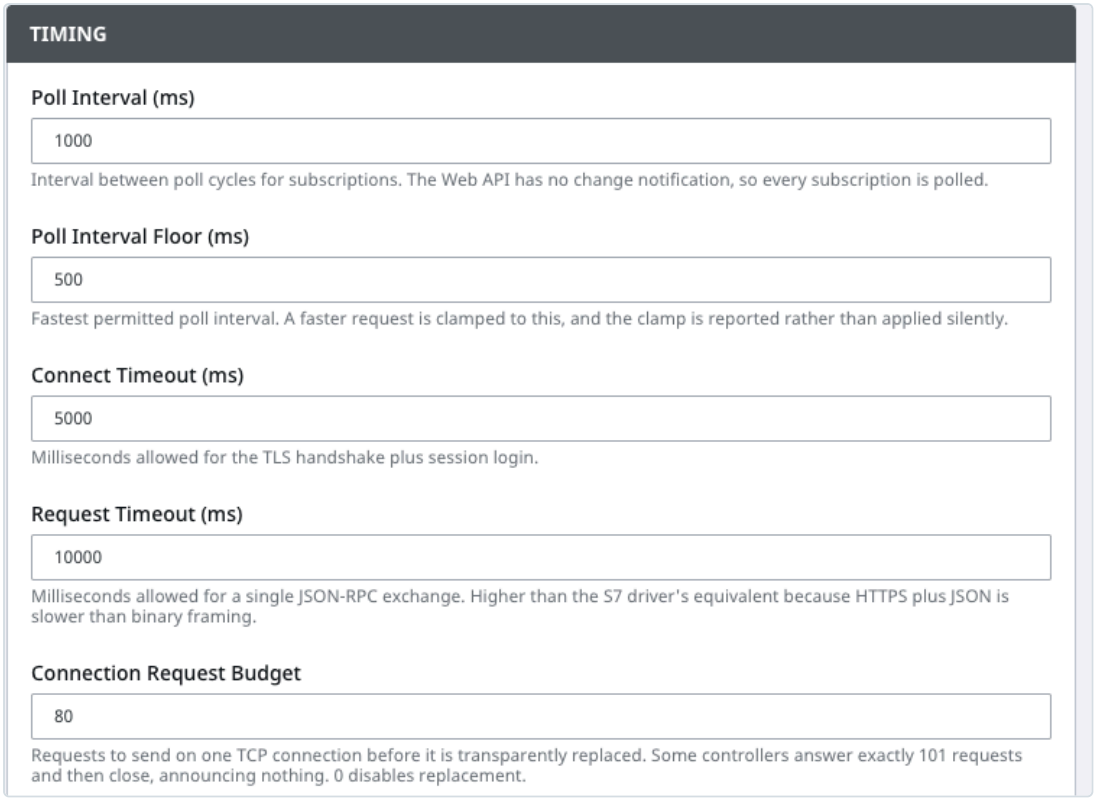
1. **Gepinnte Fingerprints** — die richtige Antwort für ein Werkszertifikat. Sicher, und funktioniert, ohne die Prüfung abzuschalten. Schlägt eine Prüfung fehl, enthält die Fehlermeldung den tatsächlichen Fingerprint der Gegenstelle, bereit zum Einfügen.
2. **Ein eigener Trust Store** — für ein von der eigenen CA signiertes Zertifikat.
3. **Die Vertrauensanker der Plattform** — für ein Zertifikat einer öffentlichen CA.
4. **`Verify Certificate` aus** — Verschlüsselung ohne Authentifizierung. Nur für die Entwicklung.

Lassen Sie TLS Version leer, sofern Sie keinen Grund haben. Leer bedeutet: TLS 1.3 bevorzugt, mit Rückfall auf TLS 1.2, den ältere Steuerungen benötigen. Eine hier festgelegte Version nimmt diesen Rückfall fort.

Timing

Einstellung	Beschreibung	Standard
Poll Interval (ms)	Abstand zwischen den Abfragezyklen für Subscriptions	`1000`
Poll Interval Floor (ms)	Kürzestes zulässiges Abfrageintervall	`500`
Connect Timeout (ms)	Zeit in Millisekunden für TLS-Handshake und Sitzungsanmeldung	`5000`
Request Timeout (ms)	Zeit in Millisekunden für einen einzelnen JSON-RPC-Austausch	`10000`
Connection Request Budget	Anfragen je TCP-Verbindung, bevor diese transparent ersetzt wird	`80`

Jede Subscription wird gepollt. Die Web API kennt keine Änderungsbenachrichtigung, *Poll Interval* ist für dieses Gerät also die tatsächliche Datenrate — kein Rückfallwert. Ein Wert unterhalb von *Poll Interval Floor* führt nicht zum Fehler: der Treiber begrenzt die Anfrage auf den Mindestwert und meldet die Begrenzung, statt sie stillschweigend anzuwenden. Das Formular weist Sie darauf hin, wenn das eingetragene Intervall unter dem Mindestwert liegt.



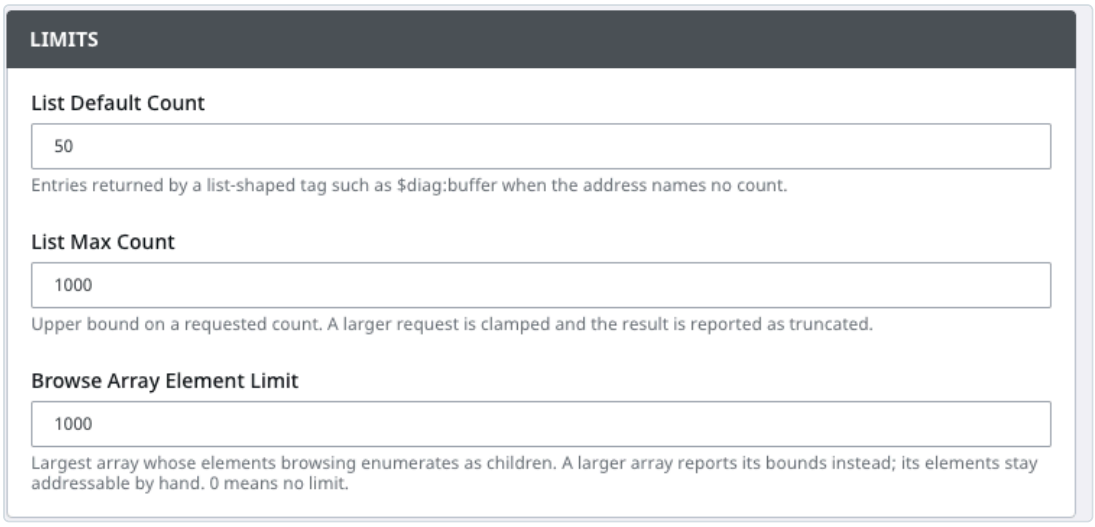
Request Timeout liegt bewusst höher als beim S7-Treiber, weil HTTPS mit JSON langsamer ist als binäre Rahmen. Ein JSON-RPC-Austausch, der mehrere Sekunden dauert, ist kein Fehler.

Connection Request Budget gibt es wegen eines realen Verhaltens mancher Steuerungen: einzelne Firmware-Stände beantworten genau 101 Anfragen auf einer Verbindung und schließen sie dann kommentarlos. Die Verbindung nach 80 Anfragen zu ersetzen verhindert, dass das je als fehlgeschlagener Lesevorgang sichtbar wird. Der Wert `0` schaltet das Ersetzen ab — sinnvoll nur bei einer Steuerung, bei der nachweislich kein Bedarf besteht.

Limits

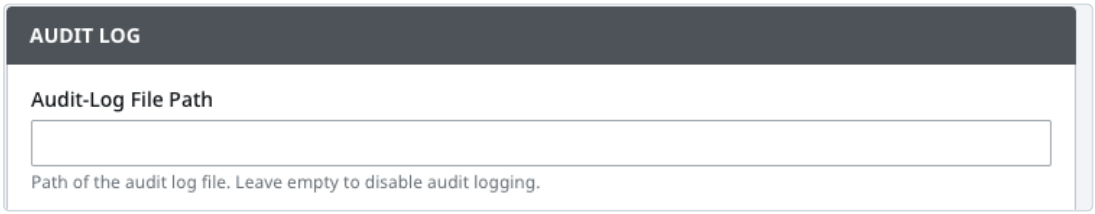
Einstellung	Beschreibung	Standard
List Default Count	Anzahl der Einträge eines listenförmigen Tags wie `$diag:buffer`, wenn die Adresse keine Anzahl nennt	`50`
List Max Count	Obergrenze einer angeforderten Anzahl. Größere Anfragen werden begrenzt und das Ergebnis als gekürzt gemeldet	`1000`
Browse Array Element Limit	Größtes Array, dessen Elemente beim Browsen als Kindknoten aufgeführt werden. `0` bedeutet keine Grenze	`1000`

Ein Array oberhalb von **Browse Array Element Limit** meldet statt Kindknoten seine Grenzen — niemals einen gekürzten Anfang, der sich wie ein schlicht kleineres Array darstellen würde. Seine Elemente bleiben von Hand adressierbar, das Array selbst ist jedoch **nicht als Ganzes lesbar**: der Lesepfad zerlegt ein Aggregat genau so wie der Browse-Baum, ein Array, das der Baum nicht aufführt, liest der Lesepfad also ebenfalls nicht.



Audit Log

Das `Audit Log` schreibt wichtige Diagnoseinformationen in eine eigene Protokolldatei. Diese Datei kann schnell wachsen; es empfiehlt sich daher, sie nur zur Fehlersuche zu aktivieren.



Einstellung	Beschreibung	Standard
Audit-Log File Path	Absoluter Pfad der Audit-Log-Datei, in die Ereignisse geschrieben werden. Leer lassen, um sie zu deaktivieren	*(leer)*

HINWEIS: Bitte stellen Sie sicher, dass der Benutzer, unter dem der `Ignition`-Prozess läuft, Schreibrechte auf diesen Pfad hat.

Tag-Adressierung

Ausschließlich symbolisch — eine absolute Adressform gibt es nicht. Die Grammatik folgt der Anzeigeform von TIA:

```
"<Datenbaustein>".<Element>[.<Element>…][:<TYP>]
"<SPS-Tag>"[:<TYP>]
$<Namensraum>:<Verb>[:<TYP>][(<Anzahl>)]
```

Adresse	Bedeutung
`"TestData".g_s16`	Ein `INT`-Element des Datenbausteins `TestData`
`"TestData".g_plant.meta.str`	Ein verschachteltes Struktur-Element
`"OPC_UA_DB"."OPC Data".g_b1`	Ein Element, dessen Name kein einfacher Bezeichner ist — **es muss in Anführungszeichen stehen**
`"ManualTrigger"`	Ein SPS-Tag aus der globalen Tag-Tabelle
`$diag:buffer[0..19]`	Die 20 neuesten Einträge des Diagnosepuffers

> **Adressen sind *nicht* mit dem S7-Modul austauschbar.** Jener Treiber akzeptiert neben symbolischen auch absolute Formen wie `%DB42:0.0:BOOL`; dieser ist rein symbolisch und verwendet die TIA-Anzeigeform in Anführungszeichen. Tags lassen sich nicht durch einen Wechsel des Geräts zwischen beiden verschieben.

Das Typsuffix ist optional; ohne es gilt der Typ der Steuerung selbst, was der Normalfall ist. Elemente, deren Namen etwas anderes als Buchstaben, Ziffern und `_` enthalten — Leerzeichen und Bindestriche sind in echten Projekten häufig —, müssen in Anführungszeichen stehen; das Browsen erledigt das für Sie.

Gebrowste Adressen sind unverändert verwendbar. Das Browsen meldet außerdem Array-Grenzen (`ARRAY [1..8] OF BOOL`), Zeichenkettenkapazität (`STRING(40)`) und Schreibschutz — alles aus den Metadaten der Steuerung.

Diagnose über synthetische Tags

Bereiche außerhalb der Tags werden über einen reservierten `$`-Namensraum auf dem gewöhnlichen Lese-/Schreibpfad erreicht — der eigentliche Grund, dieses Modul zu wählen:

Tag	Richtung	Bedeutung
`$diag:buffer` / `$diag:buffer[0..49]`	lesen	Diagnosepuffer, neueste zuerst
`$diag:alarms`	lesen	Aktive Alarme

Tag	Richtung	Bedeutung
`$control:operating-mode:STRING`	lesen / schreiben	Betriebsart lesen oder `run` / `stop` schreiben, um sie zu ändern
`$session:permissions`	lesen	Effektive Rechte des Kontos

Eine Liste sagt, wenn sie gekürzt wurde. `$diag:buffer` und `$diag:alarms` liefern höchstens **List Max Count** Einträge, und **List Default Count**, wenn die Adresse keinen Bereich nennt. Der Ordner jeder Liste enthält deshalb drei weitere Variablen, die mit der Liste gelesen werden und keine eigene Anfrage kosten:

Variable	Typ	Bedeutung
`<Liste>#list-truncated`	Boolean	`true`, wenn die Steuerung mehr Einträge hatte, als zurückkamen
`<Liste>#list-entries-omitted`	Int32	Wie viele ausgelassen wurden; `0` bei einer vollständigen Liste
`<Liste>#list-entries-available`	Int32	Die Anzahl der Steuerung; leer bei einer vollständigen Liste, die sie nicht meldet

zum Beispiel `$diag:buffer#list-truncated`. Der Ordner **Management** liest den Puffer als `$diag:buffer`; er enthält also **List Default Count** Einträge und meldet `true`, sobald die Steuerung mehr hat. Ein Bereich, den Sie selbst adressieren, etwa `$diag:buffer[0..19]`, kommt wie angefragt zurück und wird erst oberhalb von **List Max Count** gekürzt. Erhöhen Sie **List Default Count** (und **List Max Count** mit), wenn Sie den Rest brauchen.

Eine Variable, die buchstäblich mit `$` beginnt, kann nicht adressiert werden — der Namensraum ist reserviert.

> **Ein Schreibvorgang auf `$control:operating-mode` wirkt auf die Anlage, und die Bestätigung bedeutet nicht, wonach sie aussieht.** Ein `OK` heißt, dass die Anforderung **eingereiht** wurde, nicht dass die Betriebsart gewechselt hat — und eine zweite Anforderung wird ebenfalls eingereiht, nicht abgelehnt. An einer S7-1200 G2 gemessen: drei widersprüchliche Befehle im Abstand von 8–10 ms (`run`, `stop`, `run`) wurden jeweils bestätigt und **alle drei der Reihe nach ausgeführt**; die CPU wurde in rund 135 ms gestoppt und wieder gestartet, ohne dass an irgendeiner Stelle ein Fehler gemeldet wurde.
>
> - **Wiederholen Sie einen Betriebsartwechsel niemals, weil die Bestätigung auf sich warten lässt.** Der erste ist bereits eingereiht; eine Wiederholung ist eine zweite Ausführung.
> - **Sichern Sie die Bedienoberfläche ab.** Ein doppelt geklicktes STOP wird zweimal ausgeführt.
> - **Bestätigen Sie durch Lesen, nicht über die Quittung.** Ein Lesevorgang unmittelbar nach dem Schreiben meldet die *alte* Betriebsart, was von einer Ablehnung nicht zu unterscheiden ist. Pollen Sie, bis sich der Wert einpendelt, und werten Sie eine Zeitüberschreitung als Fehlschlag.

Arrays und Strukturen

Aggregate werden als Ganzes gelesen, aber nie als Ganzes geschrieben. Die Leitung kennt keine Aggregatübertragung: ein Array über seinen Namen zu lesen wird von der Steuerung abgelehnt, eine Struktur ebenso. Der Treiber bedient Lesevorgänge dennoch, indem er das Aggregat in die Adressen zerlegt, die die Steuerung *akzeptiert*, und die Antwort wieder zusammensetzt. Schreibvorgänge bleiben bewusst abgelehnt — ein zerlegter Lesevorgang, der auf halbem Weg scheitert, ist eine wiederholbare schlechte Beobachtung, ein zerlegter Schreibvorgang dagegen hinterlässt in der Steuerung eine Mischung aus neuen und alten Elementen ohne Rücknahmemöglichkeit. **Schreiben Sie Elemente einzeln.**

Ein fehlgeschlagenes Element lässt das gesamte Tag fehlschlagen, und die Meldung nennt es; eine Struktur mit Lücken sähe vollständig aus und trüge still `null`-Werte.

Drei Details sind wichtig, wenn Sie Indizes von Hand zusammensetzen statt zu browsen:

- **Indizes stehen kommagetrennt in einer Klammergruppe.** `g_matI16_2x3[1,1]` funktioniert; `g_matI16_2x3[1][1]` wird als ungültige Adresse abgelehnt.
- **Die deklarierte untere Grenze bleibt erhalten und wird nie normalisiert.** Im selben Baustein kann ein Array bei 0 und ein anderes bei 1 beginnen. Eine angenommene Basis adressiert das falsche Element.
- **Eine Zwischenzeile eines mehrdimensionalen Arrays ist selbst ein Array** — durch Zerlegung lesbar, nicht schreibbar.

Über **Browse Queries** steuern Sie, welche Symbole beim Browsen im Ignition Designer erscheinen.

Tag-Gruppen (Tag Groups)

Standardmäßig werden alle Tags in der Standard-Tag-Gruppe erfasst, das heißt alle ausgewählten Tags werden im Standardintervall von einmal pro Sekunde abgefragt.

Werden unterschiedliche Intervalle benötigt, legen Sie weitere Tag-Gruppen an, konfigurieren dort die gewünschten Intervalle und weisen die Tags den entsprechenden Gruppen zu.

Bei diesem Treiber wirkt das Intervall der Tag-Gruppe mit *Poll Interval* zusammen. Die Web API wird gepollt; eine Tag-Gruppe, die schneller ist als die Steuerung liefern kann, erzeugt keine schnelleren Daten, sondern Anfragen, die sich stauen. Bemessen Sie die Tag-Gruppen danach, was die Steuerung beantworten kann, nicht danach, was wünschenswert wäre.

Lizenzdiagnose

Meldet ein Gerät ein Lizenzproblem, erklärt das Modul die Ursache direkt am Gateway. Öffnen Sie die Einstellungen des Geräts (**Config > OPC UA > Device Connections**, Gerät bearbeiten oder neu anlegen): Die Karte **Licence** oben im Formular zeigt die Diagnose. Sie erscheint auch beim Anlegen eines neuen Geräts, sodass sich die Lizenz prüfen lässt, bevor das erste Gerät existiert.

Die Diagnose zeigt:

- **welches Gateway** es ist, und seine Rolle, wenn es Teil eines redundanten Paares ist;
- **die Ignition-Lizenz** dieses Moduls auf diesem Gateway: aktiviert, Testphase läuft (mit ihrem Ende) oder Testphase abgelaufen;
- **welche ToddySoft-Lizenzdaten angekommen sind**: wie viele Teile, ob sie vollständig sind, und eine Prüfsumme, die die Lizenz kennzeichnet, ohne sie preiszugeben;
- das **Urteil des Treibers** und die **Ursache** in Worten.

Ursache	Bedeutung	Was tun
`IGNITION_UNLICENSED`	Ignition wendet auf diesem Gateway keine aktivierte Lizenz auf dieses Modul an (Testphase läuft oder ist abgelaufen). ToddySoft-Lizenzdaten kommen nur mit einer aktivierten Lizenz an.	Einen Lizenzschlüssel aktivieren, der dieses Modul auf diesem Gateway abdeckt. In einem redundanten Paar braucht jeder Knoten seine eigene Aktivierung. Ist ein Schlüssel aktiviert und diese Ursache bleibt, siehe Fehlerbehebung: Ignition beurteilt das Modul unter Umständen nach einem älteren, abgelaufenen Schlüssel, der es ebenfalls abdeckt.
`NO_DATA_FOR_DRIVER`	Der aktivierte Schlüssel enthält keine ToddySoft-Lizenz für diesen Treiber.	Den Bericht an ToddySoft senden.
`DATA_INCOMPLETE`	Die Lizenzdaten sind unvollständig oder beschädigt angekommen.	Den Bericht an ToddySoft senden; die Lizenz muss neu geliefert werden.
`DATA_REJECTED`	Der Treiber hat die gelieferte Lizenz abgelehnt; der Grund wird angezeigt.	Den Bericht an ToddySoft senden.
`RUNNING_ON_BUNDLED_TRIAL`	Eine Lizenz wurde geliefert, aber nicht angenommen; der Treiber läuft mit seiner eingebauten Testlizenz, bis diese endet.	Den Bericht an ToddySoft senden.

Ursache	Bedeutung	Was tun
`NOT_CHECKED`	Lizenzdaten sind angekommen, aber der Treiber hat kein Urteil abgegeben – etwa weil er nicht geladen werden konnte.	Den Bericht an ToddySoft senden.
`VALID`	Lizenziert; Stufe und Ablaufdatum werden angezeigt.	Nichts.
`NOT_YET_KNOWN`	Die ersten Sekunden nach dem Start, bevor Ignition einen Lizenzstatus meldet.	Kurz warten.

Einen Bericht an den Support senden. Die Diagnose gibt es als Klartext-Bericht – das, was bei jeder Lizenzfrage mitzuschicken ist. **Copy report** legt ihn in die Zwischenablage; **Show report** zeigt ihn an. Derselbe Text steht angemeldeten Benutzern unter `http://<gateway>:<port>/data/com.toddysoft.connect.s7webapi/licence-report` bereit. Er enthält keine Lizenzdaten, keine Schlüssel, keine Zugangsdaten und keine Kundendaten. In einem redundanten Paar schicken Sie den Bericht beider Knoten.

Das Gateway-Log enthält denselben Bericht einmal bei jeder Änderung der Lizenzsituation.

Fehlerbehebung

Lizenzprobleme nach dem Aktivieren eines neuen oder verlängerten Lizenzschlüssels — `DATA_INCOMPLETE` mit „duplicate fragment index" oder `IGNITION_UNLICENSED` mit „trial expired" in der Lizenzdiagnose:

- **Am Gateway sind gleichzeitig zwei Lizenzschlüssel aktiviert, die dieses Modul abdecken**, meist ein verlängerter oder erweiterter Schlüssel neben dem alten. Ignition übergibt dem Modul die Lizenzdaten beider Schlüssel in nummerierten Teilen mit denselben Nummern; das Modul kann nicht erkennen, welche Teile zusammengehören, behält von jedem Paar einen, und die zusammengesetzte Lizenz ist unlesbar. Je nachdem, welche Teile es behält, werden Geräte abgewiesen oder laufen mit der eingebauten Testlizenz des Moduls, bis diese abläuft.

- Ist einer der beiden Schlüssel **abgelaufen**, beurteilt Ignition das Modul unter Umständen allein nach diesem: Die Diagnose zeigt `IGNITION_UNLICENSED` („trial expired - no activated licence applies to this module") und „Data: none received", obwohl der neue Schlüssel erfolgreich aktiviert wurde.
- Pro Gateway genau **einen** aktivierten Lizenzschlüssel behalten, der dieses Modul abdeckt: den alten unter **Platform > Licensing** deaktivieren und anschließend das Modul (oder das Gateway) neu starten. In einem redundanten Paar jeden Knoten prüfen.

Das Modul lädt nicht, oder im Log erscheinen `ClassNotFoundException` bzw. `NoClassDefFoundError`:

- Das **OPC-UA**-Modul von Ignition fehlt oder ist deaktiviert. Dieser Treiber ist ein OPC-UA-Gerätetreiber und kann ohne dieses Modul nicht geladen werden. Installieren bzw. aktivieren Sie es unter **Config > System > Modules** und starten Sie das Gateway neu — siehe Installation

Das Gerät zeigt „Faulted":

- Prüfen, ob Hostname/IP der Steuerung über den konfigurierten HTTPS-Port erreichbar ist
- Prüfen, ob die **Web API** für die CPU im TIA-Projekt aktiviert ist
- Gateway-Logs prüfen: **Status > Diagnostics > Logs**

Die Anmeldung schlägt fehl, obwohl das Passwort stimmt:

- Dem Konto fehlt höchstwahrscheinlich die **Web-API-Berechtigung**, die im TIA-Projekt getrennt vom Webserver-Zugriff vergeben wird. Die TLS-Ebene gelingt, die Anmeldung wird abgewiesen — das liest sich genau wie ein falsches Passwort.

Die TLS-Verbindung schlägt fehl:

- Ein werkseitig selbstsigniertes Zertifikat kann eine gewöhnliche Prüfung nicht bestehen. Fingerprint unter **Pinned Fingerprints** pinnen; die Fehlermeldung meldet den tatsächlichen Fingerprint der Steuerung
- Bei einem von der eigenen CA signierten Zertifikat **Trust Store Path** und **Trust Store Type** setzen
- Bei einer älteren Steuerung **TLS Version** leeren, damit der Rückfall auf TLS 1.2 wieder zur Verfügung steht

Die Logs zeigen, dass die Verbindung wiederholt geöffnet und geschlossen wird:

- Erwartet. Die Steuerung schließt untätige Verbindungen binnen Sekunden, der Treiber baut sie deshalb je Vorgang auf, statt eine offen zu halten. Das ist kein Fehler und erfordert keine Änderung an der Konfiguration.

Lesevorgänge schlagen nach etwa hundert Anfragen fehl:

- Einzelne Firmware-Stände beantworten genau 101 Anfragen auf einer Verbindung und schließen sie dann kommentarlos. **Connection Request Budget** (Standard `80`) ersetzt die Verbindung, bevor das eintritt; wurde der Wert auf `0` gesetzt, stellen Sie den Standard wieder her.

Das Lesen eines Arrays oder einer Struktur liefert einen Fehler:

- Das *Schreiben* ganzer Aggregate wird bewusst nicht unterstützt — schreiben Sie Elemente einzeln. Schlägt ein *Lesevorgang* fehl, ist ein Element gescheitert und die Meldung nennt es. Ein Array oberhalb von **Browse Array Element Limit** ist überhaupt nicht als Ganzes lesbar; seine Elemente bleiben einzeln lesbar.

Die Erfassung ist zu langsam:

- Dieser Treiber ist je Tag etwa um den Faktor zehn langsamer als der S7-Treiber und kennt keine Änderungsbenachrichtigung. Kommt es auf die Zykluszeit an, ist es das falsche Modul für die Aufgabe — verwenden Sie für Prozessdaten das S7-Modul und dieses hier für die Diagnose.

Docker-bezogene Probleme

Gewöhnliche Verbindungen funktionieren aus einem Container heraus normal: die Web API ist eine ausgehende HTTPS-Verbindung, für die diese Seite keine Identität ankündigt. Betroffen ist allein **Discover Devices**: sein Broadcast verlässt das Docker-Netz nie. Siehe Betrieb in Docker: der Discovery-Agent.

VPN-bezogene Probleme

Eine über VPN erreichte entfernte Steuerung erfordert von diesem Treiber nichts Besonderes. Schlägt die Verbindung fehl, ist es gewöhnliche Erreichbarkeit — prüfen Sie, ob der HTTPS-Port geroutet und nicht blockiert ist.

Versionshistorie

Unveröffentlicht — 1.3.0

Basiert auf ToddySoft Connect 1.3.0.

- **Neu:** dieses Handbuch.
- **Neu:** Lizenzdiagnose – warum ein Gerät lizenziert ist oder nicht, direkt am Gateway, mit einem Bericht für den Support. Siehe Lizenzdiagnose.
- **Neu:** Hinweis zur Fehlerbehebung, wenn an einem Gateway zwei Lizenzschlüssel gleichzeitig aktiviert sind, die dieses Modul abdecken. Siehe Fehlerbehebung.
- **Neu:** Gerätesuche, wenn Ignition in Docker läuft, über den ToddySoft Discovery-Agenten. Siehe Betrieb in Docker: der Discovery-Agent.

- **Neu:** Eine Diagnoseliste (`$diag:buffer` , `$diag:alarms`) sagt, wenn sie gekürzt wurde: drei Variablen in ihrem Ordner, `#list-truncated` , `#list-entries-omitted` und `#list-entries-available` .
- **Geändert:** Datums- und Zeit-Tags haben neue Werttypen. DATE und DATE_AND_TIME sind DateTime-Werte (bisher ein DateTime-Tag, der Epoch-Millisekunden als Zahl enthielt); TIME und LTIME sind Millisekunden, TIME_OF_DAY und LTIME_OF_DAY Millisekunden seit Mitternacht (Int64). Schreibzugriffe nehmen dieselben Formen an. Skripte und Bindings, die die alten Werte verwenden, müssen angepasst werden.
- **Behoben:** Ein Schreibzugriff, den die SPS ablehnt, schlägt jetzt auch in Ignition fehl, mit dem Grund als Status (zum Beispiel `Bad_TypeMismatch` für einen Wert des falschen Typs). Bisher wurde jeder Schreibzugriff als erfolgreich gemeldet, und die Ablehnung stand nur im Gateway-Log.
- **Behoben:** Ein Tag, der noch nicht gelesen wurde oder dessen letzter Lesezugriff fehlschlug, liest nicht mehr 0 mit guter Qualität (Good). Vor dem ersten Lesen hat er keinen Wert und die Qualität Uncertain; nach einem fehlgeschlagenen Lesen behält er seinen letzten Wert mit schlechter Qualität (Bad) bis zum nächsten erfolgreichen Lesen.
- **Behoben:** Wird der letzte Tag einer Tag-Gruppe (oder Abfragerate) entfernt, zeigt das Gerät nicht mehr *Connection Lost*. Ein Lesezugriff, der in diesem Moment noch lief, ließ das Gerät mit `Batch '...' not found` im Gateway-Log fehlschlagen, und es blieb bis zu einem Neustart in diesem Zustand.
- **Geändert:** Ein durchsuchtes Array ist ein Ordner, benannt nach dem Array und seinem Indexbereich (zum Beispiel `arr[0..9]`), der seine Elemente enthält; bisher lagen die Elemente direkt neben den übrigen Knoten. Ein mehrdimensionales Array, das bisher beim Durchsuchen fehlte, erscheint als verschachtelte Ordner bis zu seinen Elementen (`cube[1..2][1..2]` , darin `cube[1][1..2]` , darin `cube[1][1]`). Ein Array von Strukturen zeigt je Element einen Ordner mit dessen Komponenten (zum Beispiel `pts[2].x`). Jedes Element behält seinen OPC-Item-Pfad, bestehende Tags funktionieren also weiter.